RAG MASTERY SERIES · CLASS 3 OF 6 · AMANAILAB.COM

RETRIEVAL TECHNIQUES

Class 3: Finding the Right Information Every Time

Dense, Sparse, Hybrid, Reranking, MMR & Query Optimization

WHAT YOU WILL LEARN IN THIS CLASS:

- Dense Retrieval — Semantic vector search explained in depth
- Sparse Retrieval — BM25 keyword matching & when it's better than vectors
- Hybrid Retrieval — Combining dense + sparse (production best practice)
- Reranking — Cross-encoders that boost precision by 20-40%
- MMR — Maximum Marginal Relevance for diverse, non-redundant results
- Query Optimization — Rewriting, expansion & multi-query techniques

C1 Foundation	C2 Indexing	**C3 Retrieval**	C4 Generation	C5 Advanced	C6 Production

6-Class RAG Mastery Roadmap — You Are Here: Class 3

Class 3 — Table of Contents

3.1 | Recap & Why Retrieval Matters

The bridge between indexing and generation

In Class 2, we indexed our documents — they're now chunks stored as vectors in a database. But a knowledge base sitting idle is useless. **Retrieval** is the process of **finding the right chunks** when a user asks a question. It's the bridge between your indexed knowledge and the LLM's answer.

Why Retrieval is the Make-or-Break Step

If you retrieve the WRONG chunks → the LLM generates a WRONG answer (garbage in, garbage out).
If you retrieve the RIGHT chunks → the LLM generates an ACCURATE, cited answer.

The LLM is only as good as the context it receives. No amount of prompt engineering can fix bad retrieval.

This is why retrieval quality is the single biggest factor in RAG system performance — more important than the LLM choice, prompt design, or any other component.

Where We Are in the Pipeline

Phase 1: INDEXING ■ (Class 2 — Done)
Documents → Chunk → Embed → Store in Vector DB

Phase 2: QUERYING (Today — Class 3 focuses on the RETRIEVAL step)
User Question → Embed → ★ RETRIEVE TOP-K CHUNKS ★ → Build Prompt → LLM → Answer

Today we master everything inside that ★ star ★ — the retrieval step.

3.2 | Dense Retrieval (Vector Search)

Semantic search — finding meaning, not just words

What is Dense Retrieval?

Dense retrieval uses embedding models to convert both the query and stored chunks into dense vectors (lists of 768-3072 numbers), then finds the chunks whose vectors are most **similar** to the query vector using cosine similarity. It understands **meaning**, not just matching keywords.

Why It's Called 'Dense'

Dense vs Sparse — The Name Explained

DENSE vectors: Every dimension has a non-zero value.
Example: [0.023, -0.891, 0.445, 0.112, -0.337, ...] → 1536 numbers, ALL non-zero.
Created by: Neural network embedding models (OpenAI, BGE, etc.)

SPARSE vectors: Most dimensions are zero, only a few have values.
Example: [0, 0, 0, 2.4, 0, 0, 0, 1.8, 0, 0, ...] → 50,000+ dims, 99% are zero.
Created by: BM25/TF-IDF (keyword frequency counting)

Dense = packed with meaning in every number. Sparse = mostly empty, only keyword positions filled.

The Superpower of Dense Retrieval — Semantic Understanding

User Query	Document Chunk	Same Words?	Dense Finds It?	Why?
'vacation days'	'Annual leave entitlement is 24 days'	NO	YES	Embedding understands vacation = leave = time off
'WFH policy'	'Remote work guidelines for employees'	NO	YES	WFH and remote work are semantically identical
'salary hike process'	'Annual compensation revision procedure'	NO	YES	Salary hike = compensation revision in meaning
'can I bring my dog to office'	'Pet policy: animals in the workplace'	NO	YES	Dog = pet = animal, office = workplace in embedding space
'maternity benefits'	'Parental leave and childcare support'	PARTIAL	YES	Maternity ≈ parental, benefits ≈ support semantically

Dense retrieval finds relevant documents even when the user and the document use completely different words. This is impossible with keyword search.

3.3 | How Dense Retrieval Works

Step-by-step with a real example

Scenario

Knowledge base: 847 chunks from AmanCorp HR Policy
User asks: 'How many vacation days do I get per year?'

Step	What Happens	Result
1. Embed the Query	The SAME embedding model used during indexing converts the query into a vector.	Query: 'How many vacation days do I get per year?' → Vector: [0.78, 0.82, 0.15, 0.91, ...] (1536 numbers)
2. Compare Against All Chunks	The vector database computes cosine similarity between the query vector and ALL 847 stored chunk vectors.	847 similarity scores calculated in ~5 milliseconds (thanks to HNSW index)
3. Rank by Similarity	All 847 chunks are ranked from highest to lowest similarity score.	Chunk #142: 0.94 (annual leave) Chunk #143: 0.89 (leave carry forward) Chunk #287: 0.72 (sick leave) Chunk #501: 0.31 (parking policy) ...
4. Return Top-K	The top K chunks (typically K=3 to 6) with highest scores are returned.	Returns chunks #142, #143, and #287 These become the 'context' for the LLM
5. Apply Threshold (Optional)	Optionally discard chunks below a minimum similarity score (e.g., 0.70).	Chunk #501 (0.31) is discarded — too low relevance. Only high-quality chunks pass through.

Speed

Vector databases with HNSW indexing can search through 10 million vectors in under 50 milliseconds. This is why RAG feels instantaneous to users — the retrieval step is incredibly fast.

3.4 | Dense Retrieval — Strengths & Weaknesses

When it works brilliantly and when it fails

Strengths

- **Semantic understanding:** Finds relevant docs even with zero keyword overlap ('vacation' finds 'annual leave')
- **Handles paraphrasing:** Users ask questions in many different ways — dense handles all of them
- **Cross-lingual potential:** Multilingual embedding models can match English queries to Hindi documents
- **Extremely fast:** HNSW indexes search millions of vectors in milliseconds

Weaknesses — When Dense Retrieval FAILS

Failure Case	Why It Fails	Example
Exact IDs / Codes	Embeddings capture MEANING, not exact character sequences. 'SKU-4521' has no semantic meaning to an embedding model.	Query: 'price of SKU-4521' Dense returns chunks about pricing in general, NOT the specific SKU-4521 product.
Rare Technical Terms	If a term wasn't in the embedding model's training data, it gets a generic vector.	Query: 'DDSFIX-1298 status' Dense can't match this internal ticket ID meaningfully.
Exact Phrase Matching	User wants an EXACT phrase, not semantically similar content.	Query: 'Section 3.2 clause (b)' Dense may return Section 3.1 or 3.3 instead — semantically similar but WRONG section.
Negation	Embeddings struggle with negation — 'not refundable' and 'refundable' have similar embeddings.	Query: 'products that are NOT refundable' Dense may return refundable products too.
Numbers & Dates	Embedding models treat numbers as tokens, not quantities. '24 days' and '12 days' may have similar embeddings.	Query: 'policy updated after January 2025' Dense can't reliably filter by date.

Key Insight

Dense retrieval is excellent for MEANING-based questions but fails for EXACT MATCH queries. This is exactly why we need Sparse retrieval (BM25) alongside it — which is the topic of the next section.

3.5 | Sparse Retrieval (BM25 / Keyword)

Classic keyword matching — still powerful in 2026

What is Sparse Retrieval?

Sparse retrieval finds documents based on **exact keyword matching**. The most popular algorithm is **BM25** (Best Matching 25), which scores documents based on how many query terms appear in them, how frequently they appear, and how rare those terms are across all documents. It's been the backbone of search engines (Google, Elasticsearch) for 30+ years.

Why It's Called 'Sparse'

BM25 represents documents as vectors where each dimension corresponds to a word in the vocabulary. Since a document only contains a tiny fraction of all possible words, most dimensions are zero — hence 'sparse'. A vocabulary of 50,000 words means 50,000-dimensional vectors where 99%+ are zero.

When Sparse Retrieval Beats Dense

Query Type	Why Sparse Wins	Example
Product codes / IDs	BM25 does exact character matching — 'SKU-4521' matches 'SKU-4521' precisely	Query: 'SKU-4521 return policy' → BM25 finds the exact product page
Technical jargon	Specialized terms that embedding models may not understand well	Query: 'DDSFIX-1298 resolution' → BM25 finds the exact ticket
Names & proper nouns	Names are arbitrary strings — embeddings can't reason about them	Query: 'Marcos Franco email' → BM25 matches the exact name
Exact section references	Users want a specific section, not a similar one	Query: 'Section 3.2(b) leave policy' → BM25 finds '3.2(b)' exactly
Short, specific queries	When the query IS the answer — just need to find where that term appears	Query: 'ALPHA_WINDOW parameter' → BM25 finds the exact config variable

3.6 | BM25 Scoring — How It Works

TF, IDF & document length — the math made simple

BM25 scores each document based on three factors. You don't need to memorize the formula, but understanding the **intuition** behind each factor is essential for interviews:

Factor	What It Measures	Intuition	Example
TF (Term Frequency)	How many times the query term appears in the document	A document mentioning 'leave' 5 times is probably more about leave than one mentioning it once.	Doc A: 'leave' appears 5 times → higher TF Doc B: 'leave' appears 1 time → lower TF
IDF (Inverse Document Frequency)	How rare the query term is across ALL documents	Rare words are more informative. 'maternity' appearing in only 3 docs is more useful than 'the' appearing in all 847 docs.	'maternity' in 3/847 docs → high IDF (rare, valuable) 'employee' in 800/847 docs → low IDF (common, less useful)
Document Length Normalization	Adjusts score based on document length	A short document mentioning 'leave' 3 times is more focused than a long document mentioning it 3 times among 1000 other words.	Short chunk (200 chars) with 'leave' 3x → high score Long chunk (2000 chars) with 'leave' 3x → lower score

```
// BM25 Formula (for reference)

BM25 Score = SUM over each query term:
              IDF(term) × TF(term, doc) × (k1 + 1)
            ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            TF(term, doc) + k1 × (1 - b + b × docLen/avgDocLen)

Where:
  k1 = 1.2 (term frequency saturation — how quickly TF stops mattering)
  b  = 0.75 (document length normalization weight)

Don't memorize this formula. Understand the intuition:
  → Rare terms (high IDF) contribute more to the score
  → Frequent terms (high TF) contribute more, but with diminishing returns
  → Shorter documents get a boost over longer ones
```

3.7 | Sparse vs Dense — Head-to-Head

10 real queries showing where each wins

#	Query	Dense (Vector) Wins?	Sparse (BM25) Wins?	Why?
1	'vacation days per year'	■ YES	■ NO	Dense understands vacation=leave. BM25 can't find 'vacation' in 'annual leave' doc.
2	'SKU-4521 price'	■ NO	■ YES	SKU-4521 is an exact code. BM25 matches it character-by-character.
3	'WFH guidelines'	■ YES	■ NO	Dense maps WFH to 'remote work policy'. BM25 only finds 'WFH' literally.
4	'Section 3.2 clause b'	■ NO	■ YES	Exact section reference. BM25 matches '3.2' precisely.
5	'maternity benefits'	■ YES	■ BOTH	Both work — the word 'maternity' exists in the doc AND meaning matches.
6	'DDSFIX-1298 status'	■ NO	■ YES	Internal ticket ID — only exact match works.
7	'time off for new parents'	■ YES	■ NO	Dense understands this = parental leave. BM25 can't make that connection.
8	'employee handbook page 12'	■ NO	■ YES	Exact reference to page number. BM25 matches '12' in metadata.
9	'salary hike process'	■ YES	■ NO	Dense maps 'salary hike' to 'compensation revision'. BM25 misses it.
10	'Marcos Franco contact'	■ NO	■ YES	Person name — only exact keyword match works reliably.

Score: Dense 5, Sparse 6, Both 1

Neither method is universally better! Dense excels at meaning-based queries. Sparse excels at exact-match queries. The conclusion is obvious:

USE BOTH TOGETHER = Hybrid Retrieval (next section).

3.8 | Hybrid Retrieval

Dense + Sparse combined — THE production best practice

What is Hybrid Retrieval?

Hybrid retrieval runs BOTH dense (vector) and sparse (BM25) searches in parallel, then **combines their scores** using a weighted formula. This gives you the best of both worlds: semantic understanding FROM dense + exact keyword matching FROM sparse.

How It Works — Step by Step

Step	What Happens	Example
1. Run Dense Search	Embed query → cosine similarity against all vectors → get top-20 dense results	Query: 'SKU-4521 return policy' Dense returns chunks about return/refund policies (semantic match)
2. Run Sparse Search	BM25 keyword match against all chunks → get top-20 sparse results	Same query Sparse returns chunks containing 'SKU-4521' exactly (keyword match)
3. Normalize Scores	Convert both sets of scores to the same 0-1 scale (since BM25 and cosine have different ranges)	Dense scores: 0.82, 0.78, 0.71, ... BM25 scores: 12.4, 8.2, 6.1, ... → normalize to 0-1
4. Combine with Weights	Final score = α × dense_score + (1-α) × sparse_score Typical: α=0.6 (60% dense + 40% sparse)	Chunk about SKU-4521 return policy: Dense: 0.75 + Sparse: 0.92 Final: 0.6×0.75 + 0.4×0.92 = 0.818
5. Rerank & Return Top-K	Sort by combined score, return top-K chunks	The chunk mentioning BOTH 'SKU-4521' AND 'return policy' gets the highest combined score — perfect!

Choosing the Weight (α)

Weight Setting	When to Use	Example Use Case
α = 0.7 (70% dense, 30% sparse)	Most content is natural language. Users ask conceptual questions.	HR policy Q&A;, legal document search, customer support
α = 0.5 (50/50 balanced)	Mixed queries — some conceptual, some exact keyword.	E-commerce product search (names + descriptions)
α = 0.3 (30% dense, 70% sparse)	Most queries involve specific codes, IDs, or exact terms.	IT ticket search, inventory lookup, code search

Production Rule

Start with α=0.6 (60% dense, 40% sparse). Then tune based on your evaluation results. If RAGAS Context Precision is low, try adjusting the weight. Most production RAG systems in 2026 use hybrid retrieval — there's no reason not to.

3.9 | Reranking — The Precision Booster

Cross-encoders that improve results by 20-40%

What is Reranking?

Initial retrieval (dense or hybrid) uses **bi-encoders** — they embed query and documents separately, then compare vectors. This is fast but approximate. **Reranking** uses **cross-encoders** that process the query AND each document TOGETHER, producing a much more accurate relevance score.

Bi-Encoder vs Cross-Encoder

Aspect	Bi-Encoder (Initial Retrieval)	Cross-Encoder (Reranking)
How it works	Embeds query and document SEPARATELY, compares vectors	Processes query AND document TOGETHER through the same model
Speed	Very fast — embed once, compare with dot product	Slow — must run the full model for each query-document pair
Accuracy	Good (approximate similarity)	Excellent (fine-grained relevance scoring)
Scale	Can search millions of documents	Can only process 10-50 candidates (too slow for millions)
Analogy	Reading two book summaries and guessing if they're related	Reading both books side by side and carefully comparing them

The Reranking Pipeline

How It Fits Together

Step 1: Hybrid retrieval fetches top-20 candidates (fast, approximate)
Step 2: Cross-encoder reranker rescores all 20 candidates (slow, precise)
Step 3: Take top-5 after reranking (highest quality results)

Result: The final top-5 are MUCH more relevant than the initial top-5 from retrieval alone. Typical improvement: 20-40% better precision.

Popular Rerankers (2026)

Reranker	Provider	Cost	Quality	Best For
Rerank v3.5	Cohere	Paid API	★★★★★	Best quality. Production standard. Easy API.
bge-reranker-v2-m3	BAAI	Free / Local	★★★★■	Best free option. Multilingual. Runs locally.
ColBERTv2	Stanford	Free / Local	★★★★■	Token-level interaction. Very accurate.

Reranker	Provider	Cost	Quality	Best For
FlashRank	FlashRank	Free / Local	★★★■■	Fastest free reranker. Good for low-latency apps.
RankLLM	Various	Free / Local	★★★★■	Uses LLMs as rerankers. High quality but slow.

3.10 | MMR — Maximum Marginal Relevance

Diverse results instead of redundant ones

The Redundancy Problem

Normal top-K retrieval often returns very similar chunks — because the most similar vectors are usually about the exact same topic. If top-1 is about 'annual leave = 24 days', top-2 and top-3 might be slightly rephrased versions of the same thing. This wastes your context window.

How MMR Solves This

MMR (Maximum Marginal Relevance) selects chunks that are both **relevant to the query** AND **different from each other**. It balances relevance and diversity using a parameter λ (lambda):

```
// MMR Formula

MMR = argmax [ λ × Similarity(chunk, query) - (1-λ) × max(Similarity(chunk, already_selected)) ]

λ = 1.0 → Pure relevance (same as normal top-K, no diversity)
λ = 0.0 → Pure diversity (most different from already selected, ignores relevance)
λ = 0.5 → Balanced (good mix of relevant AND diverse) ← RECOMMENDED
```

MMR Example

	Normal Top-3 (Redundant)	MMR Top-3 (Diverse)
Chunk 1	'Annual leave is 24 days per year'	'Annual leave is 24 days per year' (most relevant)
Chunk 2	'Employees get 24 days of annual leave'	'Unused leave can be carried forward for 90 days' (related but different info)
Chunk 3	'Full-time staff receive 24 days leave'	'Leave must be applied 5 working days in advance' (different aspect)
LLM Gets	Same fact 3 times — wastes context	3 different facts — comprehensive answer!

3.11 | Metadata Filtering

Narrowing search BEFORE vector comparison

What is Metadata Filtering?

Instead of searching ALL vectors in the database, metadata filtering **narrows the search space first** using structured metadata (source file, category, date, department). The vector search then runs only on the filtered subset. This is both faster AND more relevant.

Common Metadata Filters

Filter Type	What It Does	Example
Source Filter	Only search within specific documents	filter: {source: 'hr_policy.pdf'} → ignores all non-HR documents
Category Filter	Only search within a category/department	filter: {category: 'engineering'} → ignores HR, finance, marketing docs
Date Filter	Only search recent documents	filter: {date_updated: {$gte: '2025-01-01'}} → ignores outdated policies
Access Control	Only search docs the user is authorized to see	filter: {access_level: ['public', 'engineering']} → respects data permissions
Language Filter	Only search in the user's language	filter: {language: 'en'} → ignores French/Spanish documents

Why Filtering Matters — A Real Example

Without filter: User asks 'leave policy'. Vector DB searches 50,000 chunks from ALL departments. Returns chunks from old 2019 policy, internal memos, and the current 2024 policy mixed together.

With filter: {source: 'hr_policy_2024.pdf'}
Vector DB searches only 847 chunks from the current HR policy. Returns only relevant, current results.

Filtering improves both SPEED (smaller search space) and QUALITY (no irrelevant sources).

3.12 | Query Optimization Techniques

Rewriting, expansion, HyDE & multi-query strategies

Users often ask vague, ambiguous, or poorly worded questions. Query optimization techniques transform the raw query into a better search query before retrieval happens.

Technique	How It Works	Example	Best For
Query Rewriting	LLM rewrites the vague query into a clear, specific search query	Input: 'PL rules' Rewritten: 'Privilege Leave policy and entitlement for employees'	Short, ambiguous, or jargon-heavy queries
Query Expansion	Add synonyms and related terms to the query to improve recall	Input: 'vacation policy' Expanded: 'vacation policy OR annual leave OR time off OR PTO'	When recall (finding all relevant docs) is critical
HyDE (Hypothetical Document Embedding)	LLM generates a hypothetical answer, which is then embedded instead of the query. The hypothesis matches document style better.	Input: 'side effects?' HyDE generates: 'Common adverse reactions include nausea, dizziness...' This hypothesis embeds closer to medical documents.	Domain-specific language mismatch between user queries and documents
Multi-Query	Generate multiple different versions of the query and search with each, then merge results	Input: 'leave policy' Query 1: 'annual leave entitlement' Query 2: 'vacation days rules' Query 3: 'paid time off policy'	Complex questions where one query might miss relevant docs
Step-Back Prompting	Ask a broader version of the question first to get general context, then ask the specific question	Input: 'Can I carry forward leave from Q3?' Step-back: 'What is the leave carry-forward policy?' Then: specific Q3 answer from retrieved context.	Specific questions that require general context first

3.13 | Retrieval Pipeline — Putting It All Together

Complete production retrieval flow

The Complete Retrieval Pipeline (Production-Grade)

Step	Component	What Happens	Why
1	Query Rewriting	LLM rewrites vague query into a clear search query	Fixes ambiguous/short user queries
2	Metadata Filtering	Filter vector DB by source, category, date, access level	Reduces search space, improves relevance
3	Hybrid Search	Run dense (vector) + sparse (BM25) in parallel	Catches both semantic AND keyword matches
4	Score Fusion	Combine dense + sparse scores with weights (e.g., 60/40)	Single ranked list from both methods
5	Reranking	Cross-encoder rescores top-20 candidates	Boosts precision by 20-40%
6	MMR Diversity	Select top-K that are relevant AND diverse	Avoids redundant chunks in context
7	Return to LLM	Pass top 3-5 chunks as context for generation	LLM receives the highest quality context

You Don't Need ALL Steps for Every Project

Minimum viable retrieval (start here):
→ Dense retrieval with top-K = 4

Good retrieval (most projects):
→ Hybrid retrieval (dense + BM25) with metadata filtering

Production-grade retrieval (enterprise):
→ Query rewriting → Metadata filtering → Hybrid search → Reranking → MMR

Start simple, add complexity only when evaluation (RAGAS) shows you need it.

3.14 | Common Retrieval Mistakes

Top 8 failures with exact fixes

#	Mistake	What Goes Wrong	How to Fix
1	Using only dense retrieval	Misses exact keyword matches (product codes, IDs, names)	Add BM25 for hybrid retrieval. Weight: 60% dense + 40% sparse.
2	K too small (k=1 or 2)	Only one chunk retrieved — misses part of the answer if it spans multiple chunks	Start with k=4-6. Increase for complex multi-part questions.
3	K too large (k=20)	Too many irrelevant chunks pollute the LLM context. LLM gets confused by noise.	Reduce to k=4-6. Use reranking to pick best 5 from top 20.
4	No similarity threshold	Low-relevance chunks (score 0.3) are included alongside high-relevance chunks (score 0.9)	Set minimum threshold at 0.70. Discard anything below.
5	No metadata filtering	Retrieves from wrong documents or outdated versions	Always filter by source, category, or date before vector search.
6	No reranking	Initial retrieval is approximate — the top-1 result may not be the most relevant	Add Cohere Rerank or BGE Reranker. Fetch 20, rerank to top 5.
7	No query optimization	Vague queries like 'leave' return random leave-related chunks	Add query rewriting: 'leave' → 'employee annual leave entitlement policy'.
8	Not evaluating retrieval	No idea if the right chunks are being retrieved	Use RAGAS Context Precision and Context Recall. Test with 20-50 questions weekly.

3.15 | Summary & What's Next

Key takeaways + preview of Class 4

Class 3 — Key Takeaways

- **Retrieval = finding the right chunks** for the user's query. If retrieval fails, the LLM answer will be wrong.
- **Dense retrieval** (vector search) understands MEANING — 'vacation' finds 'annual leave'. But fails on exact codes/IDs.
- **Sparse retrieval** (BM25) matches exact KEYWORDS — perfect for codes, names, section numbers. But misses synonyms.
- **Hybrid retrieval** combines both (60% dense + 40% sparse). This is the production best practice — always use it.
- **Reranking** (cross-encoder) rescores top-20 candidates for much better precision. Improves results by 20-40%.
- **MMR** ensures retrieved chunks are diverse — avoids sending the LLM 5 chunks that say the same thing.
- **Metadata filtering** narrows search space before vector comparison — faster and more relevant.
- **Query optimization** (rewriting, HyDE, multi-query) fixes vague/ambiguous user queries before searching.
- **Production pipeline**: Query rewrite → Filter → Hybrid search → Rerank → MMR → Top-K to LLM.
- **Start simple** (dense with k=4), add complexity only when RAGAS evaluation shows you need it.

Preview of Class 4: Generation & Prompt Design

What We'll Cover in Class 4

Now that we can retrieve the right chunks, how do we generate the best answer? Class 4 covers:

• Prompt Anatomy — the 3 essential parts of a RAG prompt
• System Prompt Engineering — preventing hallucination with careful instructions
• Chain Types — Stuff, Map-Reduce, Refine, Map-Rerank explained
• Citation Strategies — how to make the LLM cite sources properly
• Handling 'I Don't Know' — when the context doesn't contain the answer
• Full End-to-End Build — complete HR Chatbot from scratch

Make sure you understand Retrieval before moving to Generation!

6-Class Roadmap

Class	Title	Status
Class 1	Foundation — What, Why & How RAG Works	■ Completed
Class 2	Indexing Pipeline — Chunking, Embeddings, Vector DB	■ Completed

Class	Title	Status
Class 3 (TODAY)	Retrieval — Dense, Sparse, Hybrid, Reranking, MMR	■ Completed
Class 4	Generation & Prompt Design — Chains, Citations, Full Build	■ Next
Class 5	Advanced RAG — HyDE, CRAG, Self-RAG, GraphRAG, Agentic	■ Coming
Class 6	Production & Interview Prep — RAGAS, Deployment, 20 Q&A;	■ Coming